

CV SCREENER RAG

AI-Powered Recruitment Assistant
Architecture & Implementation



Author AI Engineer (Technical Assessment)

Date July 2026

Version 1.0.0

Status Complete

TABLE OF CONTENTS

- 1. Executive Summary 3
- 2. System Architecture 4
- 3. Component Diagram 5
- 4. Data Flow 6
- 5. Technology Stack 7
- 6. Implementation Details 8
- 7. API Reference 10
- 8. Project Structure 11
- 9. Security & Privacy 12
- 10. Scalability Path 13
- 11. Evaluation Criteria 14

1. EXECUTIVE SUMMARY

The **CV Screener RAG** is an AI-powered recruitment assistant that enables natural-language queries over a pool of candidate resumes. Built for the Leadtech Full-Stack AI Engineer technical assessment, it implements a complete **Retrieval-Augmented Generation (RAG)** pipeline with semantic search, LLM-based answer synthesis, and source attribution.

Key Capabilities

- **Chat Interface** -- Ask questions about candidates in natural language
- **Semantic Search** -- Find relevant CV chunks using vector embeddings
- **LLM Generation** -- Synthesize grounded answers with source badges
- **Relevance Filtering** -- Strict keyword validation prevents false positives
- **Privacy-Safe** -- All 30 CVs are synthetically generated (zero real PII)

Architecture Highlights

- **Modular** -- Separate modules for data, PDF, RAG, API, and UI
- **Local-first** -- Embeddings on CPU (MiniLM), vector store persistent (ChromaDB)
- **Triple fallback** -- OpenRouter -> Google AI Studio -> Retrieval-only
- **Zero-build frontend** -- Vanilla HTML/CSS/JS, no npm/webpack
- **Centralized strings** -- All text in data/strings.json for easy localization

2. SYSTEM ARCHITECTURE

Five layers with clear separation of concerns. Each layer is independent and swappable.

#1 Data -- Synthetic CV generation (30 PDFs) with Faker + ReportLab. All data in editable JSON.

#2 Ingestion -- PDF extraction -> Section parsing -> Contextualized chunking -> Embedding -> ChromaDB.

#3 Retrieval -- Query embedding -> Semantic search -> Strict keyword filtering -> Top-K relevant chunks.

#4 Generation -- System prompt (grounding rules) + Context injection -> LLM -> Source extraction.

#5 Presentation -- FastAPI REST + Vanilla HTML/JS chat UI with source badges and responsive design.

3. COMPONENT DIAGRAM

High-level system architecture showing all components and their interactions.

```
+-----+ | PRESENTATION LAYER | |
+-----+ | | Frontend (index.html + chat.js + chat.css)
| | | Chat Input | Messages | Source Badges | Examples | |
+-----+
+-----+ | HTTP/JSON v
+-----+ | API LAYER (FastAPI) | | +-----+
+-----+ +-----+ +-----+ | | /api/chat | /api/stats | /api/ingest | /api/config | | |
(async) | |(metrics) | |(re-index) | |(diagnostics)| | | +-----+ +-----+ +-----+
+-----+ | +-----+ | v
+-----+ | CORE ENGINE (RAGEngine) | |
+-----+ +-----+ +-----+ | | INGESTION | | RETRIEVAL | | GENERATION |
| | | PDF->Text-> | | Query->Embed-> | | Prompt+LLM | | | Chunks->Embed | | ChromaDB->Filter | | + Sources
| | | ->Store | | ->Top-K | | | | +-----+ +-----+ +-----+
+-----+ | v v v +-----+
+-----+ +-----+ +-----+ | ChromaDB | | Sentence | | OpenRouter / | | (VectorStore) | |
Transformers | | Google AI Studio | | Persistent | | all-MiniLM | | (Cloud LLM) | | Cosine sim | | 384-dim,
CPU | | Free tier | +-----+ +-----+ +-----+
```

Figure 1: Five-layer architecture with component interactions.

4. DATA FLOW

4.1 Ingestion Flow

Step	Action	Output
1	CV Generation -- Faker (seed=42)	30 PDFs with avatars, sections, skills
2	Text Extraction -- pdfplumber	Raw text with layout preserved
3	Section Detection -- Regex patterns	{SUMMARY, EXPERIENCE, EDUCATION, ...}
4	Contextualized Chunking	[Name] [Section] + content prefix
5	Embedding -- all-MiniLM-L6-v2	Vector[384] per chunk
6	Vector Storage -- ChromaDB	Persistent HNSW index

4.2 Query Flow

Step	Action	Output
1	User Input	Natural language question
2	Query Embedding	Vector[384] via MiniLM
3	Semantic Retrieval	Top-15 chunks from ChromaDB
4	Relevance Filtering	Score = matched / total keywords; discard 0s
5	Prompt Construction	System rules + User context + Question
6	LLM Generation	OpenRouter -> Google -> Retrieval-only fallback
7	Response Formatting	Answer + Source badges + Scores

5. TECHNOLOGY STACK

Layer	Technology	Purpose	Why
CV Gen	ReportLab + Pillow + Faker	PDF creation	Deterministic, privacy-safe, professional
PDF Parse	pdfplumber	Text extraction	Better layout than PyPDF2
Embeddings	sentence-transformers (MiniLM)	Semantic vectors	384-dim, ~80MB, CPU-only, free
Vector DB	ChromaDB	Persistent storage	Zero-config, Python-native, HNSW
LLM	OpenRouter (openrouter/free)	Cloud inference	Free tier, auto-routing
Fallback	Google AI Studio (Gemini)	Secondary LLM	Also free, higher limits
Backend	FastAPI + Uvicorn	REST API	Async, auto-docs, Pydantic
Frontend	Vanilla HTML/CSS/JS	Chat UI	Zero build step, portable
Data	JSON files	Configuration	Editable without code changes

6. IMPLEMENTATION DETAILS

6.1 Synthetic CV Generator (`generate_cvs.py`)

Reads `data/cv_data.json`. Creates 30 PDFs with avatars, sections, skills, education. Faker seed=42 ensures reproducibility.

6.2 PDF Processor (`pdf_processor.py`)

Uses `pdfplumber` for layout-preserving extraction. Regex-based section detection. Returns structured `CVDocument` objects.

6.3 RAG Engine (`rag_engine.py`)

Three pipelines: (1) Ingestion -- contextualized chunking [Name][Section] + MiniLM + ChromaDB; (2) Retrieval -- semantic search + strict keyword filtering; (3) Generation -- system prompt + OpenRouter->Google->retrieval fallback.

6.4 FastAPI Backend (`main.py`)

8 REST endpoints. Auto-ingests CVs on startup. Prints LLM provider diagnostics. Swagger UI at `/docs`.

6.5 Chat Frontend (`index.html` + `chat.js` + `chat.css`)

Zero-build single-page app. Real-time messaging, source badges, example questions, auto-resize textarea, loading animation, responsive design.

6.6 Strings Management (`strings_loader.py` + `data/strings.json`)

All UI text, prompts, logs, errors centralized in JSON. Cached loading with format string support. Ready for i18n.

7. API REFERENCE

Method	Endpoint	Description	Auth
GET	/	Serve chat frontend	None
POST	/api/chat	Main RAG query (async)	None
GET	/api/stats	Vector store statistics	None
POST	/api/ingest	Trigger CV re-indexing	None
GET	/api/candidates	Parsed candidate metadata	None
GET	/api/config	LLM provider diagnostics	None
GET	/health	Health check	None
GET	/docs	Swagger UI (auto-generated)	None

8. PROJECT STRUCTURE

The project is organized into logical directories with clear separation of concerns.

cv-screener-rag/ -- Root directory

|-- README.md -- Project overview & quick start

|-- generate_cvs.py -- Synthetic CV generator

|-- requirements.txt -- Python dependencies

|-- start.sh -- One-command startup script

|-- .env.example -- Configuration template

backend/ -- Backend modules

|-- main.py -- FastAPI app & REST endpoints

|-- rag_engine.py -- Core RAG pipeline

|-- pdf_processor.py -- PDF extraction & parsing

|-- config.py -- Environment configuration

|-- strings_loader.py -- String loading & i18n

frontend/ -- Frontend assets

|-- index.html -- Chat UI (HTML only)

|-- chat.js -- Chat logic

|-- chat.css -- Chat styles

data/ -- Data & configuration

|-- cvs/ -- 30 generated PDFs

|-- cv_data.json -- Generation data (editable)

|-- strings.json -- UI strings (editable)

docs/ -- Documentation

|-- ARCHITECTURE.md -- Technical deep-dive

|-- Project_Architecture_Documentation.pdf -- This document

vectorstore/ -- ChromaDB (created at runtime)

9. SECURITY & PRIVACY

Concern	Mitigation
Prompt injection	System prompt forbids answering from outside provided context
Hallucination	LLM must say 'I don't have that information' if context insufficient
Data leakage	All CVs are synthetic -- zero real PII used
API key exposure	.env excluded from git via .gitignore
File traversal	PDF processing restricted to data/cvs/ directory
False positives	Strict keyword filtering prevents irrelevant chunks reaching LLM
Rate limiting	Multiple LLM providers with automatic fallback chain

10. SCALABILITY PATH

Bottleneck	Current	Scale Path
Single machine	Local ChromaDB	ChromaDB server + Redis caching
LLM latency	Sync calls	SSE streaming + faster models
Concurrent users	Single worker	Request queue + gunicorn workers
Larger corpus	Local ChromaDB	PostgreSQL + pgvector or Pinecone
Multi-tenant	Single collection	Collection-per-tenant isolation
Better retrieval	Basic semantic	Hybrid BM25 + semantic + re-ranking

11. EVALUATION CRITERIA MAPPING

Mapping Leadtech evaluation criteria to implemented features.

Criterion	Response	Evidence
Execution & Functionality	End-to-end RAG works	30 CVs, 203 chunks, grounded chat
Thought Process	Documented architecture decisions	Why ChromaDB, MiniLM, manual RAG, vanilla JS
Code Quality	Modular, typed, documented	5 backend modules, 3 frontend files, .env, strings.json
Creativity & Ingenuity	Contextualized chunking, strict filtering, triple fallback	[Name][Section] prefixes, keyword scoring, fallback chain
AI Literacy	Embeddings, HNSW, prompt engineering	MiniLM, system prompts, fallback modes, relevance validation
Learn & Adapt	Iterated on bugs: regex, false positives, 429	split/join regex, removed SequenceMatcher, Google fallback

Conclusion

This project demonstrates full-stack AI engineering: data generation, vector search, LLM integration, and UI. The system is privacy-safe, reproducible, and extensible. All Leadtech requirements are implemented, with additional features including relevance filtering, triple LLM fallback, centralized string management, and comprehensive documentation.